Cheatsheet ICPC — Matemática y Trigonometría

Complemento del cheatsheet principal · probado con g++ -std=c++20 y Python 3

0. Índice

Complemento del primer cheatsheet: aquí NO se repite lo que ya está allá (mod 1e9+7 con factoriales, criba, divisores, bases, Grundy, geometría en enteros). Mismo formato: **cuándo → código → trampa**. Todo el C++ asume `#include <bits/stdc++.h>`, `using namespace std;`, `typedef long long ll;` y fue compilado y probado; varias plantillas se validaron con los ejemplos de UCAB 2025 (A, C, I, K) y LAR 2024 (J).

Código	Sección	Úsala cuando…
[F]	Flotantes	La salida pide decimales o "error absoluto o relativo"
[TR]	Trigonometría	Aparecen ángulos, grados, rotaciones, triángulos por lados
[GF]	Geometría con flotantes	Distancias, proyecciones, círculos, intersecciones
[VO]	Áreas y volúmenes	Conos, esferas, cilindros, "hasta dónde llenar"
[IV]	Intervalos	"Longitud cubierta", segmentos que se solapan
[CB]	Combinatoria con n enorme	n hasta 10^18, módulo grande o no primo
[PR]	Probabilidad	"Valor esperado", dados, "repetir hasta"
[NT]	Teoría de números	ax+by=c, sistemas de congruencias, primalidad 10^18, recurrencias
[PE]	Permutaciones	Intercambios, ciclos, aplicar una permutación k veces
[RX]	Mapa de ejemplos	Reconocer qué plantilla usar

Regla de tipo numérico (decídelo antes de escribir):

Situación	Usa
Todo es entero y cabe en 9·10^18	`ll`, sin flotantes
Un producto de dos números ≤ 10^18	`__int128` [CB1] o Python
Hay π, raíces, senos o divisiones no exactas	`long double` [F1]
Se puede comparar con enteros (a/b < c/d ⇔ a·d < c·b si b, d > 0)	Enteros: siempre preferible
Enteros gigantes o fracciones exactas, pocas operaciones	Python (`int`, `Fraction`)

[F] Flotantes

[F1] Base flotante — Cópialo arriba de cualquier problema con decimales. `long double` da ~18 cifras (`double` ~15). Trampa: nunca compares flotantes con `==`; usa `sgn`/`eq`.

```
typedef long double ld;
const ld PI = acosl(-1);
const ld EPS = 1e-9;
int sgn(ld x){ return (x > EPS) - (x < -EPS); }      // -1, 0, 1 con tolerancia
bool eq(ld a, ld b){ return fabsl(a - b) < EPS; }
```

Con `long double` usa las versiones con `l`: `sqrtl`, `fabsl`, `sinl`, `cosl`, `tanl`, `atan2l`, `acosl`, `hypotl`, `floorl`, `powl`. Leer: `ld x; cin >> x;` funciona igual.

[F2] Imprimir decimales — Cuando dicen "error absoluto o relativo ≤ 10^-6", imprime 9 cifras o más.

```
cout << fixed << setprecision(9) << x << "\n";   // ponlo una vez al inicio de main
printf("%.9Lf\n", x);                            // ld con printf: %Lf (L mayuscula)
```

```
print(f"{x:.9f}")
```

Trampa: si la salida esperada es entera ("16", "300") y el juez acepta error relativo, imprimir `16.000000000` también es correcto. Si piden un entero exacto, usa `llround(x)`.

[F3] Búsqueda binaria en reales — Cuándo: buscar x con f(x) = objetivo y f creciente (alturas, tiempos, radios). 200 iteraciones siempre alcanzan; no uses `while (hi - lo > EPS)` (puede no terminar).

```
template<class F> ld bsReal(ld lo, ld hi, ld target, F f){
    for (int it = 0; it < 200; it++){ ld mid = (lo + hi) / 2; if (f(mid) < target) lo = mid;
else hi = mid; }
    return (lo + hi) / 2; }
// uso: ld x = bsReal(0, 1e9, objetivo, [&](ld t){ return /* f(t) */; });
```

[F4] Búsqueda ternaria — Cuándo: f sube y luego baja (unimodal) y buscas el máximo. Para mínimo, cambia < por >. Trampa: con zonas planas en el medio falla.

```
template<class F> ld ternaryMax(ld lo, ld hi, F f){
    for (int it = 0; it < 200; it++){ ld m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1; else hi = m2; }
    return (lo + hi) / 2; }
```

[F5] Funciones seguras — Por redondeo, `sqrt(-1e-18)` da NaN y `acos(1.0000001)` también. Usa siempre estas versiones.

```
ld safeSqrt(ld x){ return sqrtl(max((ld)0, x)); }
ld safeAcos(ld x){ return acosl(max((ld)-1, min((ld)1, x))); }
```

[F6] División entera hacia abajo/arriba con negativos — En C++ `-7 / 2 == -3` (trunca hacia 0), no −4.

```
ll floorDiv(ll a, ll b){ return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); }
ll ceilDiv(ll a, ll b){ return a / b + ((a % b != 0) && ((a < 0) == (b < 0))); }
```

[F7] Python para flotantes y exactitud

```
import math
from fractions import Fraction
from decimal import Decimal, getcontext
math.pi; math.radians(180); math.degrees(math.pi); math.atan2(y, x); math.hypot(3, 4)
math.isclose(0.1 + 0.2, 0.3)        # True (comparar flotantes)
math.floor(-2.5), math.ceil(-2.5)   # -3, -2
-7 // 2                             # -4 (floor: redondea hacia -inf)
-(-a // b)                          # ceil(a / b) con enteros: -(-7 // 2) = 4
f = Fraction(1, 6) + Fraction(1, 3) # 1/2 exacto; f.numerator, f.denominator
getcontext().prec = 50; Decimal(2).sqrt()   # 50 cifras
```

Trampa: `round(2.675, 2)` da `2.67` (2.675 no es exacto en binario). Para imprimir usa f-strings y deja que el juez tolere el error.

[TR] Trigonometría

[TR1] Grados, radianes y ángulos — Las funciones de C++ y Python trabajan en RADIANES. Trampa: si el enunciado da grados (θ = 45), convierte antes de `sin`/`cos`/`tan`.

```
ld toRad(ld deg){ return deg * PI / 180; }
ld toDeg(ld rad){ return rad * 180 / PI; }
ld angle360(ld y, ld x){ ld a = atan2l(y, x); return a < 0 ? a + 2 * PI : a; } // [0, 2pi)
```

Función	Devuelve	Nota
`atan2l(y, x)`	Ángulo del vector (x, y) en (−π, π]	Usa esta, no `atan(y/x)` (falla con x = 0 y pierde el cuadrante)
`acosl(v)`	[0, π]	Pasa por `safeAcos` [F5]
`asinl(v)`	[−π/2, π/2]	Igual: recorta a [−1, 1]
`tanl(t)`	tan	Explota cerca de 90°

[TR2] Triángulos — Lados a, b, c; el ángulo C es el opuesto al lado c.

```
ld angleFromSides(ld a, ld b, ld c){ return safeAcos((a*a + b*b - c*c) / (2*a*b)); } //
angulo C (ley de cosenos)
ld sideFromAngle(ld a, ld b, ld C){ return safeSqrt(a*a + b*b - 2*a*b*cosl(C)); }   // lado c
ld heron(ld a, ld b, ld c){ ld s = (a + b + c) / 2; return safeSqrt(s * (s-a) * (s-b) * (s-
c)); }
ld areaSAS(ld a, ld b, ld C){ return a * b * sinl(C) / 2; }
// ley de senos: a / sin(A) = b / sin(B) = c / sin(C) = 2R  (R = radio circunscrito)
```

Necesito	Fórmula
Área con base y altura	b·h / 2
Radio circunscrito	R = a·b·c / (4·área)
Radio inscrito	r = área / s, con s = (a+b+c)/2
Altura sobre el lado c	2·área / c
Sombra de un objeto de altura h con luz a θ grados	h / tan(θ)
Triángulo equilátero de lado s	área = √3·s²/4, altura = √3·s/2

[TR3] Rotación, ángulo entre vectores, polares — Usa el struct `PT` de [GF1].

```
PT rotate(PT a, ld t){ return {a.x * cosl(t) - a.y * sinl(t), a.x * sinl(t) + a.y * cosl(t)};
}   // antihorario, t en radianes
ld angleBetween(PT a, PT b){ return fabsl(atan2l(cross(a, b), dot(a, b))); }  // [0, pi]
PT polar(ld r, ld t){ return {r * cosl(t), r * sinl(t)}; }
```

Rotar alrededor de un punto c: `rotate(p - c, t) + c`. Rotar 90° exacto sin flotantes: (x, y) → (−y, x).

[GF] Geometría con flotantes

Si todas las coordenadas son enteras y solo necesitas cruz/punto/área, usa la versión entera del primer cheatsheet ([GE]). Esta es para cuando aparecen raíces, divisiones o círculos.

[GF1] Punto / vector — Base de todo lo demás; requiere [F1].

```
struct PT { ld x, y;
    PT operator+(PT o) const { return {x + o.x, y + o.y}; }
    PT operator-(PT o) const { return {x - o.x, y - o.y}; }
    PT operator*(ld k) const { return {x * k, y * k}; }
    PT operator/(ld k) const { return {x / k, y / k}; } };
ld dot(PT a, PT b){ return a.x * b.x + a.y * b.y; }
ld cross(PT a, PT b){ return a.x * b.y - a.y * b.x; }
ld norm(PT a){ return hypotl(a.x, a.y); }
ld dist(PT a, PT b){ return norm(a - b); }
PT perp(PT a){ return {-a.y, a.x}; }                                  // rota 90 antihorario
```

[GF2] Rectas y segmentos — Una recta se da con dos puntos a, b.

```
PT project(PT p, PT a, PT b){ PT d = b - a; return a + d * (dot(p - a, d) / dot(d, d)); }
// pie de la perpendicular
PT reflect(PT p, PT a, PT b){ return project(p, a, b) * 2 - p; }
ld distLine(PT p, PT a, PT b){ return fabsl(cross(b - a, p - a)) / norm(b - a); }
ld distSeg(PT p, PT a, PT b){ PT d = b - a; ld t = dot(p - a, d) / dot(d, d);
    t = max((ld)0, min((ld)1, t)); return dist(p, a + d * t); }
bool lineInter(PT a, PT b, PT c, PT d, PT &out){ // rectas ab y cd; false si paralelas
    ld den = cross(b - a, d - c); if (sgn(den) == 0) return false;
    out = a + (b - a) * (cross(c - a, d - c) / den); return true; }
```

[GF3] Círculos — Devuelven 0, 1 o 2 puntos. Trampa: `circleCircle` con dos círculos idénticos devuelve vacío (infinitos puntos): revísalo aparte.

```
vector<PT> circleLine(PT c, ld r, PT a, PT b){ // puntos de corte circulo-recta
    PT p = project(c, a, b); ld h2 = r*r - dot(p - c, p - c);
    if (h2 < -EPS) return {}; PT d = (b - a) / norm(b - a); ld h = safeSqrt(h2);
    if (h < EPS) return {p}; return {p - d * h, p + d * h}; }
vector<PT> circleCircle(PT c1, ld r1, PT c2, ld r2){
    ld d = dist(c1, c2); if (d > r1 + r2 + EPS || d < fabsl(r1 - r2) - EPS || d < EPS) return
{};
    ld a = (r1*r1 - r2*r2 + d*d) / (2*d), h = safeSqrt(r1*r1 - a*a);
    PT m = c1 + (c2 - c1) * (a / d), off = perp(c2 - c1) * (h / d);
    if (h < EPS) return {m}; return {m - off, m + off}; }
PT circumcenter(PT a, PT b, PT c){ // centro del circulo por 3 puntos (no colineales)
    PT B = b - a, C = c - a; ld D = 2 * cross(B, C);
    return a + PT{C.y * dot(B, B) - B.y * dot(C, C), B.x * dot(C, C) - C.x * dot(B, B)} / D;
}
ld circleInterArea(ld r1, ld r2, ld d){ // area comun de dos circulos con centros a distancia
d
    if (d >= r1 + r2) return 0; if (d <= fabsl(r1 - r2)){ ld r = min(r1, r2); return PI * r *
r; }
    ld a1 = safeAcos((d*d + r1*r1 - r2*r2) / (2*d*r1)), a2 = safeAcos((d*d + r2*r2 - r1*r1) /
(2*d*r2));
    return r1*r1*(a1 - sinl(2*a1)/2) + r2*r2*(a2 - sinl(2*a2)/2); }
```

Relación de dos círculos con distancia d entre centros: separados si d > r1+r2; tangentes exteriores si d = r1+r2; se cortan si |r1−r2| < d < r1+r2; uno dentro del otro si d < |r1−r2|.

[GF4] Polígonos — Vértices en orden (horario o antihorario).

```
ld polyArea(vector<PT>& p){ ld s = 0; int n = p.size(); for (int i = 0; i < n; i++) s +=
cross(p[i], p[(i+1)%n]); return fabsl(s) / 2; }
PT centroid(vector<PT>& p){ ld A = 0; PT c = {0, 0}; int n = p.size();
    for (int i = 0; i < n; i++){ ld w = cross(p[i], p[(i+1)%n]); A += w; c = c + (p[i] +
p[(i+1)%n]) * w; }
    return c / (3 * A); }
int inPolygon(vector<PT>& p, PT q){ // 1 dentro, 0 en el borde, -1 fuera (sirve para no
convexos)
    int n = p.size(); bool in = false;
    for (int i = 0; i < n; i++){ PT a = p[i], b = p[(i+1)%n];
        if (sgn(cross(b - a, q - a)) == 0 && sgn(dot(a - q, b - q)) <= 0) return 0;
        if ((a.y > q.y) != (b.y > q.y) && q.x < a.x + (b.x - a.x) * (q.y - a.y) / (b.y -
a.y)) in = !in; }
    return in ? 1 : -1; }
ld regularArea(int n, ld s){ return n * s * s / (4 * tanl(PI / n)); }   // poligono regular,
n lados de largo s
```

Polígono regular de n lados	Fórmula
Ángulo interior	(n − 2)·180° / n
Circunradio (centro a vértice)	R = s / (2·sin(π/n))
Apotema (centro a lado)	s / (2·tan(π/n))
Área con circunradio R	n·R²·sin(2π/n) / 2
Diagonales	n·(n − 3) / 2
Triángulos con vértices del polígono	C(n, 3) → [CB2] si n es enorme

[VO] Áreas y volúmenes

[VO1] Volúmenes — Requiere [F1].

```
ld cone(ld r, ld h){ return PI * r * r * h / 3; }
ld frustum(ld r, ld R, ld h){ return PI * h * (R*R + r*r + R*r) / 3; }  // cono truncado
(radios r y R)
ld cylinder(ld r, ld h){ return PI * r * r * h; }
ld sphere(ld r){ return 4 * PI * r * r * r / 3; }
ld sphereArea(ld r){ return 4 * PI * r * r; }
ld cap(ld r, ld h){ return PI * h * h * (3 * r - h) / 3; }              // casquete esferico
de altura h
ld pyramid(ld baseArea, ld h){ return baseArea * h / 3; }
```

Figura	Área / volumen
Círculo	área πr², perímetro 2πr

Figura	Área / volumen
Sector de ángulo t (rad)	área $r^2 \cdot t/2$, arco $r \cdot t$
Segmento circular de ángulo t	$r^2 \cdot (t - \sin t)/2$
Elipse de semiejes a, b	$\pi \cdot a \cdot b$
Trapecio	(b1 + b2)·h / 2
Prisma / cilindro	área de la base · altura
Cono / pirámide	área de la base · altura / 3
Superficie lateral del cono	$\pi \cdot r \cdot g$, $g = \sqrt{(r^2 + h^2)}$
Caja con huecos verticales (cilindros)	$(N \cdot M - \Sigma \pi r_i^2) \cdot$ altura

[VO2] Altura de llenado — Cuándo: "¿hasta qué altura llenar para tener la fracción f del volumen?". Patrón general: escribe vol(y) = volumen hasta la altura y (creciente) y aplícale [F3]. Probado con los 4 ejemplos de UCAB 2025 C (Café).

```
ld fillHeight(ld r, ld R, ld h, ld f){            // taza = cono truncado, radio r abajo y R arriba
    auto vol = [&](ld y){ ld ry = r + (R - r) * y / h; return frustum(r, ry, y); };   // radio crece lineal
    return bsReal(0, h, f * vol(h), vol); }
// Cafe: cout << fixed << setprecision(9) << fillHeight(r, R, h, 0.5) << "\n";
```

Trampa: el "nivel de agua al meter objetos" es volumen / área libre de la base; compara `V >= A * areaLibre` en `long double` en vez de dividir.

[IV] Intervalos

[IV1] Longitud de la unión — Cuándo: "cuánto del eje queda cubierto" (sombras, rangos de luz, tramos). O(n log n). Probado con los 3 ejemplos de LAR 2024 J (Jigsaw of Shadows), donde cada intervalo es [X, X + H / tan(θ)].

```
ld unionLength(vector<pair<ld,ld>> v){
    sort(v.begin(), v.end()); ld tot = 0, curL = 0, curR = -1e30; bool open = false;
    for (auto [l, r] : v){
        if (!open || l > curR){ if (open) tot += curR - curL; curL = l; curR = r; open = true; }
        else curR = max(curR, r); }
    if (open) tot += curR - curL; return tot; }
// Jigsaw: for (...) v.push_back({X, X + H / tanl(toRad(theta))});
```

[IV2] Fusionar intervalos enteros — Devuelve la lista de bloques disjuntos. Por defecto une [1,3] y [4,6] (se tocan); quita el `+ 1` si no deben unirse.

```
vector<pair<ll,ll>> mergeIntervals(vector<pair<ll,ll>> v){
    sort(v.begin(), v.end()); vector<pair<ll,ll>> res;
    for (auto [l, r] : v){ if (!res.empty() && l <= res.back().second + 1) res.back().second = max(res.back().second, r); else res.push_back({l, r}); }
    return res; }
```

Otros patrones de intervalos: máximo de intervalos solapados en un punto → eventos (+1 en l, −1 en r+1), ordenar y barrer; máximo de intervalos disjuntos → ordenar por extremo derecho y tomar greedy.

[CB] Combinatoria con n enorme

¿Cuál uso para C(n, k)?

Situación	Plantilla
n ≤ 10^6, mod primo grande	[M1] del primer cheatsheet (factoriales)
n hasta 10^18, k pequeño (≤ 10^6), mod primo > k	[CB2]
mod primo pequeño (p ≤ 10^6), n y k enormes	[CB3] Lucas
mod NO primo (o sin mod), n ≤ 5000	[CB4] Pascal
Valor exacto sin mod	Python `math.comb(n, k)`

[CB1] Multiplicar con módulos de hasta ~4·10^18 — Cuándo: el módulo es como 10^18+3 (UCAB 2025 K: respuesta k^2 mod 10^18+3). Trampa: `a * b % m` con `ll` desborda si a·b > 9·10^18.

```
typedef __int128 lll;
ll mulmod(ll a, ll b, ll m){ return (ll)((lll)a * b % m); }
ll powmod(ll b, ll e, ll m){ ll r = 1 % m; b %= m; if (b < 0) b += m;
    while (e){ if (e & 1) r = mulmod(r, b, m); b = mulmod(b, b, m); e >>= 1; } return r; }
void print128(lll x){ if (x < 0){ putchar('-'); x = -x; } if (x > 9) print128(x / 10);
putchar('0' + (int)(x % 10)); }
// UCAB K: cout << mulmod(k % M, k % M, M) << "\n";  con M = 10000000000000000003LL
```

`cout` no imprime `__int128`: usa `print128` (con `printf`/`putchar`, no mezcles con `cout` sin `cout.flush()`). En Python: `k * k % (10**18 + 3)`.

[CB2] C(n, k) con n hasta 10^18 y k pequeño — Cuándo: "cuántos triángulos con vértices de un polígono de n lados" (UCAB 2025 I: C(n, 3) mod 10^9+7). O(k log p).

```
ll nCrBig(ll n, ll k, ll p){ if (k < 0 || k > n) return 0; ll num = 1, den = 1;
    for (ll i = 0; i < k; i++){ num = mulmod(num, (n - i) % p, p); den = mulmod(den, (i + 1) % p, p); }
    return mulmod(num, powmod(den, p - 2, p), p); }
```

En Python sale directo con enteros grandes: `n * (n - 1) * (n - 2) // 6 % MOD` (divide ANTES del módulo).

[CB3] Lucas (p primo pequeño)

```
ll nCrSmall(ll n, ll k, ll p){ if (k < 0 || k > n) return 0; ll r = 1;
    for (ll i = 0; i < k; i++){ r = r * ((n - i) % p) % p; r = r * powmod(i + 1, p - 2, p) % p; } return r; }
ll lucas(ll n, ll k, ll p){ if (k == 0) return 1; return nCrSmall(n % p, k % p, p) * lucas(n / p, k / p, p) % p; }
```

[CB4] Triángulo de Pascal (cualquier módulo) — O(n^2) memoria y tiempo.

```
vector<vector<ll>> pascal(int N, ll m){ vector<vector<ll>> C(N + 1, vector<ll>(N + 1, 0));
    for (int n = 0; n <= N; n++){ C[n][0] = 1 % m; for (int k = 1; k <= n; k++) C[n][k] = (C[n-1][k-1] + C[n-1][k]) % m; }
    return C; }
```

[CB5] Inclusión-exclusión — Cuándo: "cuántos de 1..N cumplen al menos una condición". Suma las intersecciones de tamaño impar y resta las de tamaño par. Ejemplo: divisibles por alguno de a[] (k ≤ 20).

```
ll countDivisible(ll N, vector<ll>& a){ int k = a.size(); ll res = 0;
    for (int mask = 1; mask < (1 << k); mask++){ ll l = 1; bool big = false;
        for (int i = 0; i < k; i++) if (mask >> i & 1){ l = l / gcd(l, a[i]); if ((lll)l * a[i] > N){ big = true; break; } l *= a[i]; }
        if (big) continue; ll c = N / l; res += (__builtin_popcount(mask) & 1) ? c : -c; }
    return res; }
// "ninguno divide" = N - countDivisible(N, a)
```

[CB6] Desarreglos y Stirling

```
vector<ll> derangements(int n, ll m){ vector<ll> D(max(n + 1, 2), 0); D[0] = 1 % m; D[1] = 0;
// nadie en su lugar
    for (int i = 2; i <= n; i++) D[i] = (ll)(i - 1) * ((D[i-1] + D[i-2]) % m) % m; return D;
}
vector<vector<ll>> stirling2(int N, ll m){ vector<vector<ll>> S(N + 1, vector<ll>(N + 1, 0));
S[0][0] = 1;
    for (int n = 1; n <= N; n++) for (int k = 1; k <= n; k++) S[n][k] = (S[n-1][k-1] + (ll)k * S[n-1][k]) % m; return S; }
// S[n][k] = repartir n elementos distintos en k grupos no vacios
```

Contar…	Fórmula
Subconjuntos de n	2^n
Ordenar n con repetidos (c_1, c_2, …)	$n! / (c_1! \cdot c_2! \cdot \ldots)$
Funciones de k en n	n^k; sobreyectivas (todo n usado): n!·S[k][n]
Pares no ordenados de n	n(n−1)/2
Desarreglos D(n)	0, 1, 2, 9, 44, 265 para n = 1..6

[PR] Probabilidad y valor esperado

Reglas que resuelven casi todo

Regla	Uso
Linealidad: E[X + Y] = E[X] + E[Y] (aunque dependan)	n tareas iguales → n · E[una tarea]
Geométrica: si cada intento sale bien con prob p, E[intentos] = 1/p	"tirar hasta que salga"
Con repetición: E = p_fin·c_fin + p_rep·(c_rep + E) → despejar E	Ver [PR1]
Dos dados de 6 caras	P(iguales) = 1/6, P(A > B) = P(B > A) = 5/12
Independientes: P(A y B) = P(A)·P(B)	
Complemento: P(al menos uno) = 1 − P(ninguno)	

[PR1] Valor esperado con "repetir" — En cada ronda, con prob `pRep` se repite gastando `cRep`; si no, termina gastando en promedio `costFinal`Promedio. Despejando E = p_fin·c_fin + p_rep·(c_rep + E). Probado con los 4 ejemplos de UCAB 2025 A.

```
typedef long double ld;
ld expectedRetry(ld pRep, ld cRep, ld costFinalPromedio){
    return (costFinalPromedio * (1 - pRep) + pRep * cRep) / (1 - pRep); }
// UCAB A (dados: empate 1/6 descansan x; si no, la tarea la hace A o B con prob 1/2 c/u):
// ld E = n * expectedRetry(1.0L/6, x, (a + b) / 2.0L);   // = n * ((a+b)/2 + x/5)
```

[PR2] Probabilidad como fracción módulo p — Cuándo: "imprime la probabilidad como P·Q^{-1} mod 10^9+7". Suma y multiplica fracciones directamente en mod.

```
const ll MD = 1e9 + 7;
ll fracMod(ll P, ll Q){ return mulmod(P % MD, powmod(Q, MD - 2, MD), MD); }   // usa [CB1]
// 1/2 -> 500000004
```

[PR3] DP de probabilidad — Cuándo: distribución de una suma o de un proceso por pasos. `dp[estado]` = probabilidad de estar ahí; cada paso reparte la probabilidad. Ejemplo: suma de n dados de c caras.

```
vector<ld> diceSum(int n, int c){ vector<ld> dp(1, 1.0L);
    for (int t = 0; t < n; t++){ vector<ld> nd(dp.size() + c, 0);
        for (size_t s = 0; s < dp.size(); s++) if (dp[s] > 0) for (int f = 1; f <= c; f++)
nd[s + f] += dp[s] / c;
        dp = nd; }
    return dp; } // dp[s] = P(suma = s)
```

Valor esperado por DP: E[estado] = Σ P(transición)·(costo + E[siguiente]). Si un estado puede volver a sí mismo, despeja como en [PR1].

[NT] Teoría de números ampliada

Varias plantillas usan `mulmod`/`powmod` y `lll` de [CB1].

[NT1] Euclides extendido — Encuentra x, y con a·x + b·y = gcd(a, b).

```
ll extgcd(ll a, ll b, ll &x, ll &y){ if (b == 0){ x = 1; y = 0; return a; }
    ll x1, y1, g = extgcd(b, a % b, x1, y1); x = y1; y = x1 - (a / b) * y1; return g; }
```

[NT2] Ecuación diofántica a·x + b·y = c — Tiene solución si y solo si gcd(a, b) divide a c. Todas las soluciones: (x + k·b/g, y − k·a/g) para k entero; elige k para que x, y ≥ 0 si lo piden.

```
bool diophantine(ll a, ll b, ll c, ll &x, ll &y){ ll g = extgcd(a, b, x, y); if (c % g)
return false;
    x = (ll)((lll)x * (c / g)); y = (ll)((lll)y * (c / g)); return true; }
```

[NT3] Inverso modular con módulo NO primo — Existe solo si gcd(a, m) = 1. Si m es primo, usa `powmod(a, m - 2, m)`.

```
ll invGeneral(ll a, ll m){ ll x, y; ll g = extgcd(((a % m) + m) % m, m, x, y); if (g != 1)
return -1; return ((x % m) + m) % m; }
```

[NT4] Teorema chino del resto — Cuándo: "x deja resto r1 al dividir entre m1 y resto r2 entre m2". Sirve aunque m1, m2 no sean coprimos. Para más ecuaciones, combínalas de a dos.

```
pair<ll,ll> crt(ll r1, ll m1, ll r2, ll m2){ ll p, q; ll g = extgcd(m1, m2, p, q);   // {x,
lcm} o {-1,-1}
    if ((r2 - r1) % g) return {-1, -1};
    ll l = m1 / g * m2; lll t = (lll)(r2 - r1) / g * p % (m2 / g);
    ll r = (ll)(((lll)r1 + (lll)m1 * t) % l); if (r < 0) r += l; return {r, l}; }
```

[NT5] Phi de Euler — phi(n) = cuántos de 1..n son coprimos con n. Propiedad: a^phi(m) ≡ 1 (mod m) si gcd(a, m) = 1, así que a^e mod m = a^(e mod phi(m)) mod m.

```
ll phi(ll n){ ll r = n; for (ll p = 2; p * p <= n; p++) if (n % p == 0){ while (n % p == 0) n
/= p; r -= r / p; } if (n > 1) r -= r / n; return r; }
vector<int> phiSieve(int n){ vector<int> ph(n + 1); iota(ph.begin(), ph.end(), 0);
    for (int i = 2; i <= n; i++) if (ph[i] == i) for (int j = i; j <= n; j += i) ph[j] -=
ph[j] / i; return ph; }
```

[NT6] Cantidad y suma de divisores en O(√n) — Cuando n = p1^e1·…·pk^ek: d(n) = (e1+1)·…·(ek+1), sigma(n) = Π (1 + p + ... + p^e). Suma de divisores propios = sigma(n) − n (perfecto / abundante / deficiente).

```
pair<ll,ll> divCountSum(ll n){ ll cnt = 1, sum = 1;
    for (ll p = 2; p * p <= n; p++) if (n % p == 0){ int e = 0; ll pk = 1, s = 1; while (n %
p == 0){ n /= p; e++; pk *= p; s += pk; } cnt *= e + 1; sum *= s; }
    if (n > 1){ cnt *= 2; sum *= n + 1; } return {cnt, sum}; }
```

[NT7] ¿Es primo? (n hasta 2^64) — Miller-Rabin determinista, O(log³ n). Usa para n > 10^12, donde la división hasta √n es lenta.

```
bool isPrime(ll n){ if (n < 2) return false;
    for (ll p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) if (n % p == 0) return n == p;
    ll d = n - 1; int s = 0; while (d % 2 == 0){ d /= 2; s++; }
    for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}){ ll x = powmod(a, d, n); if (x
== 1 || x == n - 1) continue;
        bool comp = true; for (int r = 1; r < s; r++){ x = mulmod(x, x, n); if (x == n - 1){
comp = false; break; } }
        if (comp) return false; }
    return true; }
```

[NT8] Exponenciación de matrices — Cuándo: recurrencia lineal f(n) = c1·f(n−1) + ... con n hasta 10^18. O(k³ log n) con matriz k×k. Trampa: con módulo > 3·10^9 el producto `A * B` desborda: cambia por `mulmod`.

```
typedef vector<vector<ll>> Mat;
Mat mul(const Mat& A, const Mat& B, ll m){ int n = A.size(), k = B.size(), p = B[0].size();
Mat C(n, vector<ll>(p, 0));
    for (int i = 0; i < n; i++) for (int l = 0; l < k; l++) if (A[i][l]) for (int j = 0; j <
p; j++) C[i][j] = (C[i][j] + A[i][l] * B[l][j]) % m;
    return C; }
Mat matpow(Mat A, ll e, ll m){ int n = A.size(); Mat R(n, vector<ll>(n, 0)); for (int i = 0;
i < n; i++) R[i][i] = 1 % m;
    while (e){ if (e & 1) R = mul(R, A, m); A = mul(A, A, m); e >>= 1; } return R; }
ll fib(ll n, ll m){ if (n == 0) return 0; Mat F = {{1, 1}, {1, 0}}; return matpow(F, n - 1,
m)[0][0]; } // F(1)=F(2)=1
```

[NT9] Sumas cerradas — Con `__int128` para no desbordar con n hasta 10^9 o más. `geoSum` sirve con cualquier módulo (no divide).

```
lll sum1(ll n){ return (lll)n * (n + 1) / 2; }                          // 1 + 2 + ... + n
lll sum2(ll n){ return (lll)n * (n + 1) * (2 * n + 1) / 6; }            // 1^2 + ... + n^2
lll sum3(ll n){ lll s = sum1(n); return s * s; }                         // 1^3 + ... + n^3
ll geoSum(ll r, ll n, ll m){ // 1 + r + r^2 + ... + r^(n-1) mod m
    if (n == 0) return 0; if (n % 2 == 1) return (1 + mulmod(r % m, geoSum(r, n - 1, m), m))
% m;
    ll h = geoSum(r, n / 2, m); return mulmod(h, (1 + powmod(r, n / 2, m)) % m, m); }
```

Suma de a hasta b = sum1(b) − sum1(a − 1). Progresión aritmética: n·(primero + último)/2.

[NT10] Bloques de n / i — Cuándo: Σ f(n / i) para i = 1..n con n hasta 10^12. n / i toma solo ~2√n valores distintos; cada bloque [i, j] comparte el mismo cociente q.

```
ll sumFloor(ll n){ ll s = 0; for (ll i = 1; i <= n; ){ ll q = n / i, j = n / q; s += q * (j -
i + 1); i = j + 1; } return s; }
```

[PE] Permutaciones

Permutación 0-indexada: `p[i]` = a dónde va (o qué hay en) la posición i. Si la entrada es 1-indexada, resta 1 al leer.

[PE1] Componer, invertir, ciclos, potencia — Cuándo: intercambios de filas/columnas, "aplicar la mezcla k veces", "cuántas veces hasta volver al inicio".

```cpp
vector<int> compose(const vector<int>& p, const vector<int>& q){ // (p o q)[i] = p[q[i]]: primero q, luego p
    int n = p.size(); vector<int> r(n); for (int i = 0; i < n; i++) r[i] = p[q[i]]; return r;
}
vector<int> inverse(const vector<int>& p){ int n = p.size(); vector<int> r(n); for (int i = 0; i < n; i++) r[p[i]] = i; return r; }
vector<vector<int>> cycles(const vector<int>& p){ int n = p.size(); vector<bool> vis(n);
vector<vector<int>> cs;
    for (int i = 0; i < n; i++) if (!vis[i]){ vector<int> c; for (int j = i; !vis[j]; j = p[j]){ vis[j] = true; c.push_back(j); } cs.push_back(c); }
    return cs; }
vector<int> permPow(const vector<int>& p, ll k){ // p aplicada k veces, O(n) aunque k = 1e18
    vector<int> r(p.size());
    for (auto &c : cycles(p)){ int L = c.size(); for (int i = 0; i < L; i++) r[c[i]] = c[(i + k % L) % L]; }
    return r; }
```

Pregunta	Respuesta con los ciclos
Mínimo de intercambios (swaps) para ordenar	n − (número de ciclos)
Paridad (par/impar)	(n − ciclos) % 2; cada swap cambia la paridad
Veces hasta volver a la identidad (orden)	lcm de los largos de los ciclos (puede ser enorme: usa mod o `__int128`)
¿p^k = identidad?	k es múltiplo del largo de cada ciclo

Intercambios sucesivos de filas (o columnas): lleva un arreglo `row[i]` = fila original que está ahora en la posición i, aplica cada `swap(row[I], row[J])` en orden, y al final `row` es la permutación total. Filas y columnas son independientes.

[RX] Mapa: señal en el enunciado → plantilla

[RX1] Señales

Si el enunciado dice…	Ir a
"error absoluto o relativo", salida con decimales	[F1] [F2]
"hasta qué altura / radio / tiempo para que..."	[F3] (+ [VO2])
"máximo/mínimo de una función continua" que sube y baja	[F4]
Ángulo en grados, luz, sombra, pendiente	[TR1] [TR2]
Triángulo dado por sus lados	[TR2]
Girar puntos	[TR3]
Distancia de un punto a una recta / segmento	[GF2]
Círculos que se cortan, punto dentro de círculo	[GF3]
Punto dentro de un polígono, área, centro de masa	[GF4]
Cono, esfera, cilindro, taza, tanque	[VO1] [VO2]
"longitud total cubierta"	[IV1]
n hasta 10^18 y "módulo"	[CB1] [CB2]
Módulo como 10^18+3 o producto de dos ≤ 10^18	[CB1]
"al menos uno de...", "ninguno de..."	[CB5]
"valor esperado", dados, monedas, "repetir hasta"	[PR1] [PR3]
"probabilidad módulo p"	[PR2]
a·x + b·y = c, "combinar paquetes de a y b"	[NT2]
"resto r1 al dividir entre m1 y r2 entre m2"	[NT4]
Divisores de n hasta 10^12, perfecto/abundante	[NT6]
"¿es primo?" con n hasta 10^18	[NT7]
Recurrencia con n hasta 10^18 (Fibonacci y similares)	[NT8]
Σ de n / i	[NT10]
Intercambios, mezclas repetidas, "volver al orden inicial"	[PE1]

[RX2] Problemas de ejemplo

Problema	Plantilla	Idea
UCAB 2025 A · Asignación Aleatoria	[PR1]	E por tarea = (a+b)/2 + x/5; respuesta n·E
UCAB 2025 C · Café	[VO2] [F3]	Volumen hasta y crece con y: búsqueda binaria
UCAB 2025 I · Internal Triangles	[CB2]	C(n, 3) con n hasta 10^18
UCAB 2025 K · K2	[CB1]	k^2 mod (10^18+3) con `__int128`
UCV 2025 A · Deficient/Perfect/Abundant	[NT6]	sigma(n) − n en O(√n); en C++
UCV 2025 F · Rebellious Boxes	[VO1] [F1]	Nivel = V / (área base − huecos); compara sin dividir
LAR 2024 J · Jigsaw of Shadows	[TR1] [IV1]	Sombra = H / tan(θ en radianes); unir intervalos
LAR 2024 L · Latin Squares	[PE1]	Permutación total de filas y de columnas; analizar sus ciclos
LAR 2024 F · Finding Privacy	[NT2]	Bloques de 2 y 3 (ver [M6] del primer cheatsheet)